Kryptografia

POLSKO-JAPOŃSKA AKADEMIA TECHNIK KOMPUTEROWYCH

Spis treści

Wstęp 3
Historia 4
Przygotowanie środowiska 7
Uruchamianie maszyn 8
Połączenie OpenVPN 9
Zadania 10
Zadanie 9.1: Kryptografia klasyczna 11
Zadanie 9.2: Zapis heksadecymalny 13
Zadanie 9.3: Tabela ASCII 15
Zadanie 9.4: Inne sposoby zapisu danych 17
Zadanie 9.5: Wyświetlanie danych binarnych 19
Zadanie 9.6: Funkcje skrótu 21
Zadanie 9.7: Baza md5 23
Zadanie 9.8: Operacja XOR 25
Zadanie 9.9: Algorytm AES 33
Zadanie 9.10: Algorytm RSA 36
Referencje 40

Wstęp

Student pozna podstawy historii kryptografii, nauczy się idei szyfrów podstawieniowych i przestawieniowych i funkcji skrótu. Następnie pozna współczesne metody symetrycznego (AES) i asymetrycznego (RSA) szyfrowania danych oraz zapozna się z podstawowymi błędami, których należy unikać przy ich stosowaniu.

Historia

Kryptografia ma swoje korzenie w starożytności, gdzie była stosowana do zabezpieczania komunikacji. Jednym z najbardziej znanych przykładów jest szyfr Cezara, używany przez Juliusza Cezara do szyfrowania wiadomości. Polegał on na przesunięciu każdej litery alfabetu o ustaloną liczbę miejsc. W średniowieczu rozwinięto bardziej skomplikowane metody szyfrowania, takie jak szyfr polialfabetyczny Vigenère’a, który był trudniejszy do złamania.

Historia

W epoce renesansu kryptografia stała się bardziej zaawansowana, a zainteresowanie nią wzrosło, zwłaszcza w kontekście polityki i wojny. Jednak prawdziwy przełom nastąpił w XIX wieku wraz z rozwojem telegrafu i pierwszych mechanicznych urządzeń szyfrujących. W czasie I wojny światowej używano maszyn szyfrujących, takich jak niemiecki algorytm ADFGVX.

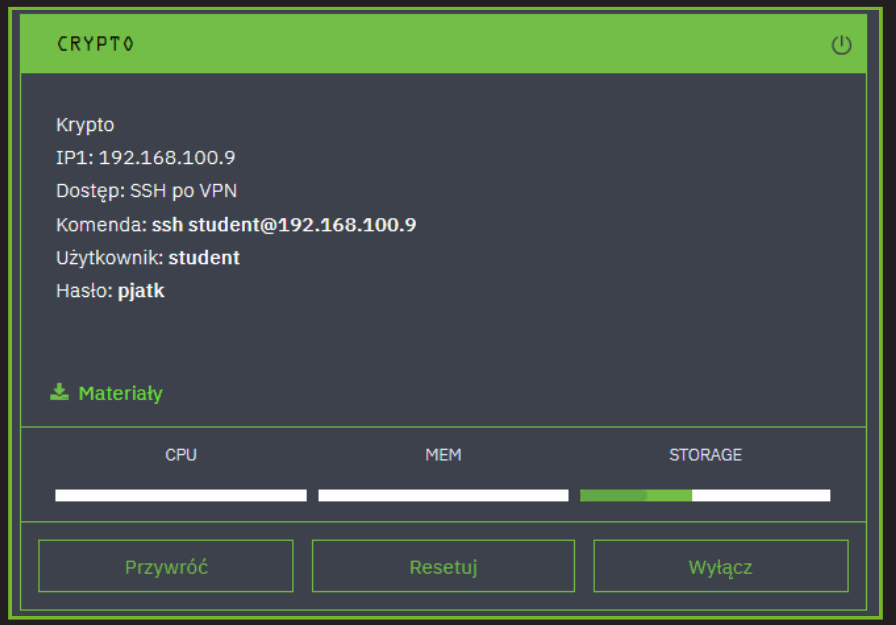
Historia

II wojna światowa przyniosła rewolucję w kryptografii, szczególnie dzięki maszynie Enigma, używanej przez Niemców. Złamanie szyfru Enigmy przez alianckich kryptologów, w tym polskich i brytyjskich, miało ogromny wpływ na wynik wojny. Prace Alana Turinga i jego zespołu w Bletchley Park doprowadziły do powstania pierwszych komputerów, co zapoczątkowało nową erę w kryptografii i technologii informacyjnej.

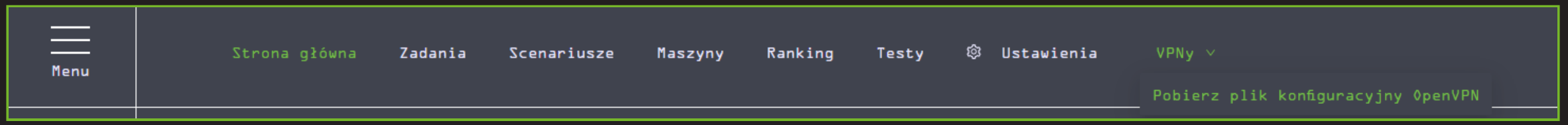
Przygotowanie środowiska

Uruchamianie maszyn

Przed rozpoczęciem upewnij się, że maszyna Crypto jest włączona.
Jeżeli coś się zepsuje, w każdej chwili możesz ją przywrócić do stanu początkowego.



Połączenie OpenVPN



Połączenie OpenVPN jest niezbędne do rozwiązania zadań.
Aby połączyć się z siecią VPN, należy pobrać plik konfiguracyjny OpenVPN z rozwijanego menu „**VPNy**” na stronie głównej.
Następnie należy wykonać komendę:

```
sudo openvpn /sciezka/do/pliku.ovpn
```

Komendy, będące częścią zadań, należy rozwiązywać w osobnej karcie/oknie terminala!

Uwaga! Wszystkie zadania z tego dokumentu znajdują się na maszynie „WSZYSTKIE ZADANIA”, a uzyskane flagi należy wkleić w formularzu na stronie.

POLSKO-JAPOŃSKA AKADEMIA TECHNIK KOMPUTEROWYCH

Zadania

Zadanie 9.1: Kryptografia klasyczna

Szyfry **podstawieniowe** i **przestawieniowe** to dwa różne podejścia do szyfrowania informacji, różniące się sposobem manipulacji tekstem jawnym.
Szyfr **podstawieniowy** polega na zastępowaniu każdej litery lub grupy liter w tekście jawnym inną literą lub grupą liter zgodnie z określonym kluczem. Przykładem jest **szyfr Cezara**, w którym każda litera w tekście jawnym jest przesunięta o określoną liczbę miejsc w alfabecie. Na przykład, przy przesunięciu o trzy miejsca, litera „A” staje się „D”, „B” staje się „E”, itd. Tekst jawny „ABC” po zaszyfrowaniu staje się „DEF”.
Szyfr **przestawieniowy**, z kolei, nie zmienia liter, ale zmienia ich kolejność w tekście jawnym.

Zadanie 9.1: Kryptografia klasyczna

Choć obecnie podobne metody szyfrowania nie są stosowane w sytuacjach, gdzie bezpieczeństwo danych jest kluczowe, a nadają się raczej do gier i zabaw, to „na rozgrzewkę” zaczniemy od złamania podobnego szyfru. W tym zadaniu użyliśmy szyfru **AtBash**.

Szyfr AtBash to starożytny szyfr **podstawieniowy**, w którym alfabet jest odwrócony, tzn. każda litera jest zastępowana swoją odpowiedniczką z końca alfabetu (A staje się Z, B staje się Y, itd.). Był używany przez Hebrajczyków, a jego prostota sprawia, że jest bardziej ciekawostką historyczną niż skutecznym narzędziem kryptograficznym. Zdekoduj zapisaną flagę, a następnie wpisz w formularzu:

`KQZGP{GlWlkrvilKlxazgvp}`

Zadanie 9.2: Zapis heksadecymalny

Ponieważ będziemy posługiwać się **danymi binarnymi**, musimy je umieć jakoś zapisać. Jednym ze sposobów zapisu danych binarnych jest zapis heksadecymalny (zwany również szesnastkowym). Należy zwrócić uwagę na różnicę między zapisem heksadecymalnym liczb, a danych binarnych. Na przykład liczba **1000**, może zostać zapisana jako **0x3e8**.

Dane binarne to z kolei ciągi bajtów, zapis **d38c9ba6af7fa8bb** oznacza więc ciąg 8 bajtów o wartościach kolejno: **211** (**0xd3**), **140** (**0x8c**), **155** (**0x9b**), **166** (**0xa6**), **175** (**0xaf**), **127** (**0x7f**), **168** (**0xa8**), **187** (**0xbb**).

Czasem zapisujac liczby w systemie heksadecymalnym pomija się prefix „0x”. Należy z kontekstu wywnioskować, czy chodzi o liczbę, czy dane binarne.

Zadanie 9.2: Zapis heksadecymalny

W zapisie danych binarnych zazwyczaj poszczególne bajty oddziela się, co znacznie ułatwia ich czytanie. Np. zamiast pisać **4fdc2d3746b412d222d2ffd61442**, możemy napisać **4f dc 2d 37 46 b4 12 d2 22 d2 ff d6 14 42**. Czasem grupuje się też bajty w pary: **4fdc 2d37 46b4 12d2 22d2 ffd6 1442**.

Zapisaliśmy flagę właśnie w tej postaci. Zdekoduj ją, a następnie wpisz w formularzu:

```
50 4A 41 54 4B 7B 73 7A 65 73 6E 61 73 63 69 65 7D
```

Możesz użyć do tego celu jednego z licznych narzędzi online, a nawet wiersza poleceń systemu Linux! Np.:

```
echo 50 4A 41 54 4B 7B 73 7A 65 73 6E 61 73 63 69 65 7D | xxd -r -p ; echo
```

Zadanie 9.3: Tabela ASCII

ASCII (czyt. aski, skrót od ang. American Standard Code for Information Interchange) - siedmiobitowy system kodowania znaków, używany we współczesnych komputerach oraz sieciach komputerowych, a także innych urządzeniach wyposażonych w mikroprocesor. Przyporządkowuje liczbom z zakresu 0-127: litery alfabetu łacińskiego języka angielskiego, cyfry, znaki przestankowe i inne symbole oraz polecenia sterujące.

Pełną tabelę możesz odnaleźć tutaj: https://www.asciitable.com/ .

Zadanie 9.3: Tabela ASCII

Tym razem zapisaliśmy flagę za pomocą kodów ASCII. Zdekoduj ją i wpisz w formularzu na stronie.

`80 74 65 84 75 123 116 64 98 101 108 64 122 110 97 75 111 119 125`

Możesz użyć do tego celu jednego z licznych narzędzi online, albo dekodować znaki ręcznie.

Zadanie 9.4: Inne sposoby zapisu danych

O pozostałych, choć niemniej ważnych metodach zapisu danych wspomnimy łącznie.

1. **Base64** - jest to metoda zapisu danych, która pozwala m.in. na przesyłanie danych binarnych w formie tekstowej. PJATK w base64 to UEpBVEs=
2. **Urlencode** - mechanizm kodowania informacji w URI, obecnie definiowany w RFC3986. Głównie jest używane do kodowania danych przesyłanych przez zapytanie GET w adresie URL. Kodowane są tylko niektóre znaki (na przykład spacja, „_”, „&”„)
3. **Encje HTML** - Sposób zapisu niektórych znaków używanych głównie jako element języka HTML. Istnieją encje nazwane, dziesiętne i szestanstkowe. Symbol „<” można zapisać jako <, &60 lub <. Pełną listę można znaleźć tutaj: https://www.w3schools.com/html/html_entities.asp .

Zadanie 9.4: Inne sposoby zapisu danych

W tym zadaniu zakodowaliśmy flagę za pomocą base64. Zdekoduj ją i wklej w formularzu na stronie:

`UEpBVEt7YTExeW91cmJhc2V9`

Możesz użyć jednego z licznych dekoderów online, a nawet wiersza poleceń systemu Linux:

```
echo UEpBVEt7YTExeW91cmJhc2V9 | base64 --decode
```

Zadanie 9.5: Wyświetlanie danych binarnych

Celem zadania jest wyświetlenie na konsoli zawartości pliku hex.bin w zapisie heksadecymalnym. Plik ten znajduje się już na maszynie testowej.
Może nas kusić użycie podstawowej komendy cat, służy ona jednak do wyświetlania danych tekstowych (spróbuj wywołać cat hex.bin i zobacz efekt). Do wyświetlenia danych binarnych można użyć np. komendy xxd:

```
xxd hex.bin
```

Zadanie zostanie zaliczone po wykonaniu powyższej komendy.

Zadanie 9.5: Wyświetlanie danych binarnych

Komenda **xxd** domyślnie wyświetla dane binarne wypisując 16 bajtów w każdej linii.
W pierwszej kolumnie znajduje się pozycja w pliku aktualnej linii. Pozycja pliku jest **liczbą** zapisaną heksadecymalnie.
W ostatniej kolumnie znajduje się **zapis tekstowy w formacie ASCII**, w którym bajty spoza zakresu ASCII oraz znaki niedrukowalne zostały zastąpione kropkami.

```
❯ , xxd hex.bin
00000000: b33d 635c 1ccb c764 9d91 88b7 6091 95bf  .=c\...d....`...
00000010: d6e9 dee2 8aad d8f5 f52e 36a2 7f9b 8d49  ..........6....I
00000020: 494e 7d93 fdbc 6d4d a4cf 36ba 49d3 76c6  IN}...mM..6.I.v.
00000030: 51b6 d3ca 33f4 7582 dcd9 5ea1 ec0d 705d  Q...3.u...^...p]
00000040: 40c4 1c21 65df 39c6 e621 44e7 ba48 b3a4  @..!e.9..!D..H..
00000050: cc0c 01e4 f278 1a26 55f7 57a1 19ef 57fc  .....x.&U.W...W.
00000060: 28d0 9e08 b551 4eb6 ac9f 2ec0 cb55 bfa0  (....QN......U..
00000070: d5f8 6ead f3e3 03f7 07b0 36e6 bd43 b2c7  ..n.......6..C..
00000080: c468 4546 0b56 54b3 ed0a f35a e4c3 5c6a  .hEF.VT....Z..\j
00000090: 0d7e 66bb 108d ecb2 a783 3dba 3a99 5646  .~f.......=.:.VF
000000a0: 4f58 6ca0 35   3b55 a334 7ba0 b055 a712  OX1.5 ;U.4{..U..
000000b0: 3887 9b0d 1073 c693 3627 f8fc 619b 7d77  8....s..6'..a.}w
000000c0: c094 7e35 5168 14c8 f7f1 ac6e d7f4 03f1  ..~5Qh.....n....
000000d0: 0e68 b86a b4ce 8ae2 104e c996 0a7d d360  .h.j.....N...}.`
000000e0: f00c 41e9 21ec 5fcc 8458 6cbd 5426 ede0  ..A.!._..Xl.T&..
000000f0: a3da c290 3d52 ac4f 7164 ffed 31ed 2ac6  ....=R.Oqd..1.*.
00000100: cd10 5285 2d2c 63a4 8c17 d813 9bf1 6f25  ..R.-,c.......o%
00000110: 144a 9b2a 2a1d 6dab 32bc ba07 209f 275a  .J.**.m.2... .'Z
00000120: 8350 e81b 8433 df98 5c4b c2d3 8a25 f014  .P...3..\K...%..
00000130: c294 ff16 4562 aa62 987c 0599 6251 1e48  ....Eb.b.|..bQ.H
00000140: bb5d be74 83b6 a0a2 4b91 ea2d aa71 811d  .].t....K..-.q..
00000150: f710 56da c426 7890 31fc 9f28 fe38 922e  ..V..&x.1..(.8..
00000160: 8777 0c0d 61bb 0bc8 2e04 a7a5 2eee 413e  .w..a.........A>
00000170: 4fde 4e9b f170 b8f0 1d58 14c9 4db6 2043  O.N..p...X..M. C
00000180: faf2 f9c8 5f94 0304 d77d d18a 455f 9ce9  ...._....}..E_..
00000190:   d2 9d76 0325 3333 3cb6 db0c f06b 8cd9    ..v.%33<....k..
000001a0: 9568 1061 a8e7 5e68 cd21 076b d677 9b43  .h.a..^h.!.k.w.C
000001b0: 1b75 3441 86ad dc2e df5a 4a8e 4af7 6b57  .u4A.....ZJ.J.kW
000001c0: 34af dea0 5588 2ae0 87c3 e3be f238 4bfa  4...U.*......8K.
000001d0: 026c 1817 779d e7a2 82f2 c837 fe50 eb4b  .l..w......7.P.K
000001e0: 3059 f9e8 2d91 19b3 af36 5f55 2e9d a1d2  0Y..-....6_U....
000001f0: 79c6 caae c641 b637 96f3 4b79 9c52 b4ed  y....A.7..Ky.R..
```

Zadanie 9.6: Funkcje skrótu

Funkcje skrótu to **algorytmy kryptograficzne**, które przekształcają dowolnie długie dane wejściowe w ciąg o stałej długości, zwany skrótem (hash). Ich cechą charakterystyczną jest to, że nawet minimalna zmiana w danych wejściowych powoduje **znaczną zmianę wyniku**, co uniemożliwia przewidzenie skrótu na podstawie oryginalnych danych. Funkcje skrótu są kluczowe w zabezpieczaniu danych, stosowane m.in. w **podpisach cyfrowych**, **weryfikacji integralności plików** oraz **przechowywaniu haseł**. Przykładami popularnych funkcji skrótu są **MD5**, **SHA-1** oraz rodzina algorytmów SHA-2 (np. **SHA-256**).

Zadanie 9.6: Funkcje skrótu

W tym zadaniu naszym celem będzie obliczenie wartości hashu MD5 dla „PJATK”. Możesz użyć do tego celu jednego z licznych narzędzi online, a nawet wiersza poleceń systemu Linux! Na przykład:

```
echo -n PJATK | md5sum
```



Zadanie 9.7: Baza md5

O ile sam algorytm MD5 jest **nieodwracalny**, tak istnieją olbrzymie bazy danych, które przechowują już przeliczone hashe, co pozwala na odzyskanie danych zapisanych w ten sposób.
Bazy danych MD5 są często wykorzystywane w atakach, gdzie można spróbować odwrócić proces haszowania, aby odzyskać oryginalne hasła. Ze względu na słabe zabezpieczenia, MD5 jest uważany za przestarzały i niewystarczający do ochrony danych w nowoczesnych systemach, gdzie zaleca się stosowanie bardziej zaawansowanych funkcji skrótu, takich jak SHA-256.

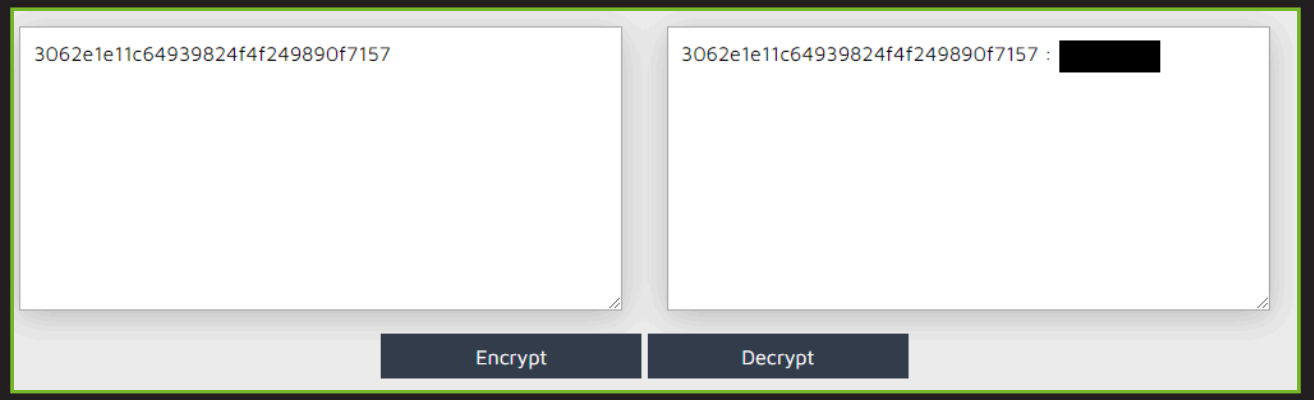
Przykładowa baza danych: https://md5decrypt.net/en/

Zadanie 9.7: Baza md5

Wykorzystamy jedną z takich baz, aby dowiedzieć się, jakie dane zostały przekształcone za pomocą MD5.

Używając wybranej bazy (może będzie trzeba sprawdzić kilka?) znajdź flagę zakodowaną jako: `3062e1e11c64939824f4f249890f7157`

Odpowiedź podaj w formie: PJATK{*flaga*} w formularzu.



Zadanie 9.8: Operacja XOR

Operacja XOR (exclusive OR) to operacja logiczna, która działa na dwóch bitach i zwraca 1, jeśli dokładnie jeden z bitów jest równy 1, a drugi 0; w przeciwnym razie zwraca 0. W edukacji szkolnej uczniowie poznają podstawowe operacje logiczne, takie jak AND, OR, NOT, a XOR jest kolejną z nich, specyficzną w swoim działaniu.

Zadanie 9.8: Operacja XOR

Własności matematyczne XOR:

```
A XOR 0 = A
A XOR A = 0
(A XOR B) XOR B = A
A XOR B = B XOR A
A XOR B = (A AND NOT B) OR (NOT A AND B)
```

Te własności czynią XOR przydatnym w szyfrowaniu, gdzie szyfrowanie można łatwo odwrócić, ale tylko jeśli zna się oryginalny klucz.

Zadanie 9.8: Operacja XOR

Spróbujmy zaszyfrować przy pomocy operacji XOR skrót PJATK za pomocą klucza abc. W tym celu zapisujemy zarówno PJATK jak i abc w postaci binarnej:

```
P: 01010000
J: 01001010
A: 01000001
T: 01010100
K: 01001011

a: 01100001
b: 01100010
c: 01100011
```

Zadanie 9.8: Operacja XOR

Teraz kolejno wykonujemy operacje (klucz powtarzamy cyklicznie, aby znaków było tyle samo, co w szyfrowanej wiadomości):

```
P (01010000) XOR a (01100001) = 00110001
J (01001010) XOR b (01100010) = 00101000
A (01000001) XOR c (01100011) = 00100010
T (01010100) XOR a (01100001) = 00110101
K (01001011) XOR b (01100010) = 00101001
```

Zadanie 9.8: Operacja XOR

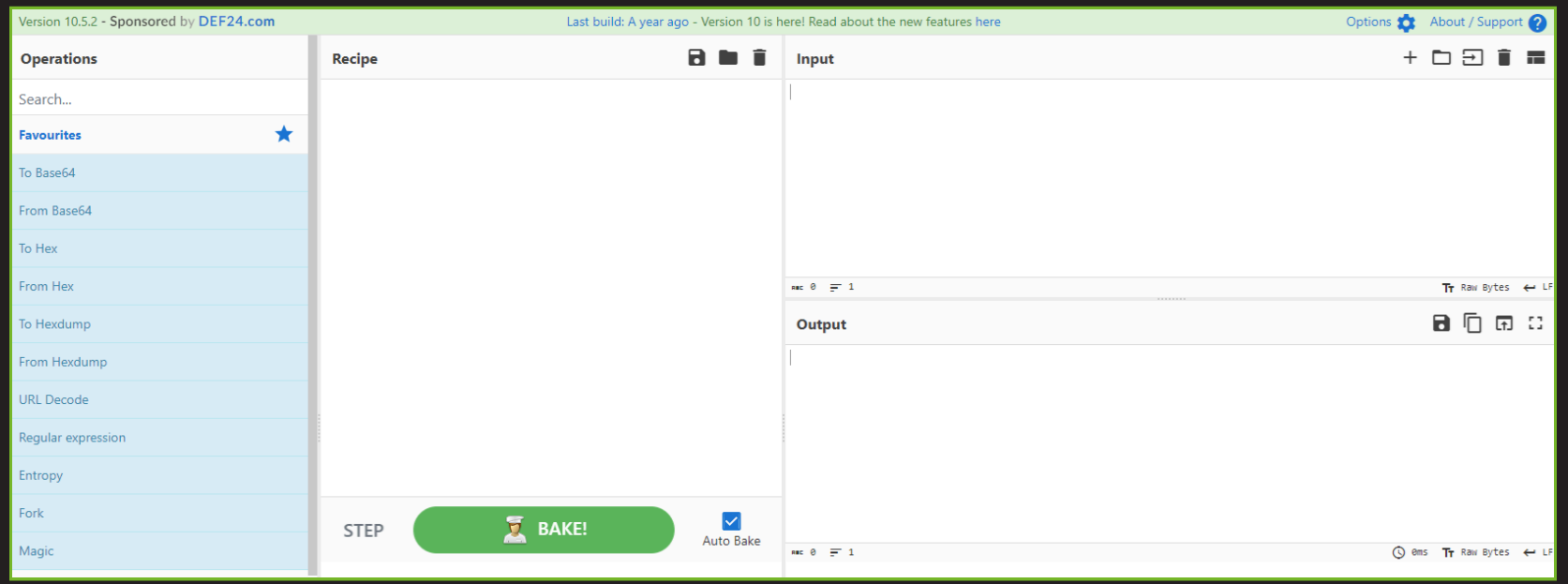
Wynikowe wartości binarne ponownie zapisujemy jako znaki ASCII:

```
00110001 (1)
00101000 (()
00100010 (")
00110101 (5)
00101001 ())
```

Po wykonaniu operacji XOR z kluczem „abc”, hasło „PJATK” zostało zaszyfrowane do ciągu binarnego, który odpowiada znakom „1("5)”. Aby odzyskać oryginalne hasło, należy ponownie zastosować operację XOR z tym samym kluczem.

Zadanie 9.8: Operacja XOR

Istnieje tylko 256 jednobajtowych kluczy XOR (jest 8 bitów w bajcie, a 2 do potęgi 8 to 256), więc możemy sprawdzić wszystkie. Wykorzystamy w tym celu narzędzie CyberChef: https://gchq.github.io/CyberChef/ .



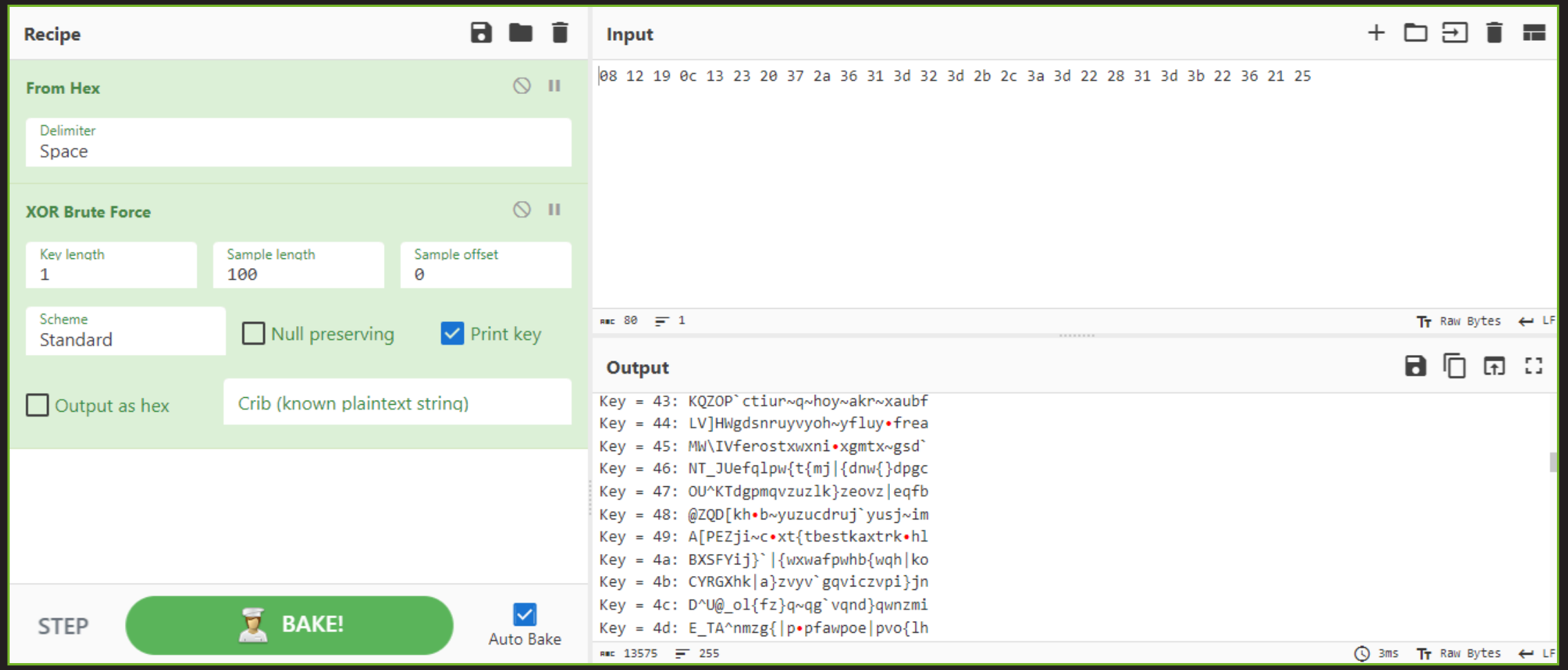
Zadanie 9.8: Operacja XOR

Celem zadania jest zdekodowanie podanego niżej ciągu znaków, który został zaszyfrowany za pomocą jednobajtowego klucza przy pomocy algorytmu XOR. Odpowiednia konfiguracja narzędzia CyberChef jest widoczna na następnym slajdzie. Ciąg do zdekodowania:

```
08 12 19 0C 13 23 20 37 2A 36 31 3D 32 3D 2B 2C 3A 3D 22 28 31 3D 3B 22 36 21 25
```

Dla jednego z kluczy powinieneś otrzymać flagę. Którego? To już musisz sprawdzić samodzielnie. Odpowiedź podaj w formularzu.

Zadanie 9.8: Operacja XOR



Zadanie 9.9: Algorytm AES

AES jest blokowym algorytmem szyfrowania **symetrycznego**. Wykorzystując go możemy zaszyfrować dane przy pomocy hasła. Zaszyfrowane dane możemy nawet opublikować w internecie, ale bez znajomości hasła nikt nie będzie w stanie odczytać niezaszyfrowanej zawartości. Jedyną możliwością odczytania zawartości jest poznanie bądź zgadnięcie hasła. AES wykonuje różne operacje matematyczne, takie jak zamiana miejscami, mieszanie, i podstawianie elementów w każdym bloku.

Zadanie 9.9: Algorytm AES

Algorytm AES może występować w wielu trybach. Z oczywistych powodów nie będziemy tu omawiać różnic, jednak przynajmniej pokażmy, ile ich jest:

-aes-128-cbc	-aes-128-cfb	-aes-128-cfb8	-aes-128-ctr	-aes-128-ecb
-aes-128-ofb	-aes-192-cbc	-aes-192-cfb	-aes-192-cfb1	-aes-192-cfb8
-aes-192-ctr	-aes-192-ecb	-aes-192-ofb	-aes-256-cbc	-aes-256-cfb
-aes-256-cfb1	-aes-256-cfb8	-aes-256-ctr	-aes-256-ecb	-aes-256-ofb
-aes128-(wrap)	-aes192-(wrap)	-aes256-(wrap)		

Zadanie 9.9: Algorytm AES

Celem zadania jest odszyfrowanie pliku plik_enc_aes-256-cbc.bin zaszyfrowanego hasłem z wykorzystaniem AES. Plik ten znajduje się na maszynie wirtualnej studenta. Aby dostać się na maszynę należy zalogować się po ssh:

```
ssh student@192.168.100.9
```

Hasło brzmi *PJATK*.

W celu odszyfrowania pliku należy wykonać komendę:

```
openssl aes-256-cbc -pbkdf2 -d -in plik_enc_aes-256-cbc.bin
```

Zdobytą w tej sposób flagę wklej w formularzu na stronie.

Zadanie 9.10: Algorytm RSA

Podobnie jak AES, RSA jest algorytmem szyfrowania. W przeciwieństwie do AES, RSA służy do szyfrowania asymetrycznego, często system RSA określa się mianem kryptografii klucza publicznego.
W RSA wykorzystuje się klucze, czyli specjalne pliki. Aby móc coś zaszyfrować, należy w pierwszej kolejności wygenerować losowy klucz prywatny. Kluczem prywatnym można dane odszyfrować. Takie wykorzystanie RSA nie różni się od AES.
Istotna część RSA leży w możliwości wygenerowania klucza publicznego związanego z zadanym kluczem prywatnym. Kluczem publicznym można dane zaszyfrować, ale nie da się nim odszyfrowywać danych. Mimo to, dane zaszyfrowane kluczem publicznym można odszyfrować wykorzystując klucz prywatny sparowany z kluczem publicznym.

Zadanie 9.10: Algorytm RSA

RSA wykorzystuje się np. do szyfrowania e-maili. Właściciel adresu e-mail może udostępnić innym swój klucz publiczny. Dzięki temu, pisząc maila do tej osoby, możemy zaszyfrować treść wiadomości przy pomocy klucza publicznego. Do odszyfrowania wiadomości potrzebny jest klucz prywatny sparowany z kluczem publicznym, a więc tylko właściciel adresu e-mail będzie w stanie odszyfrować i odczytać tę wiadomość.

Zadanie 9.10: Algorytm RSA

Celem zadania jest wygenerowanie pary kluczy RSA i użycie ich do zaszyfrowania pliku plik2.txt. Plik znajduje się na maszynie wirtualnej studenta.

Generacja klucza prywatnego:

```
openssl genrsa -out rsa_key 3072
```

Generacja klucza publicznego sparowanego z kluczem prywatnym:

```
openssl rsa -in rsa_key -pubout -out rsa_key.pub
```

Szyfrowanie:

```
openssl pkeyutl -encrypt -in plik2.txt -pubin -inkey rsa_key.pub -out plik2.bin
```

Zadanie 9.10: Algorytm RSA

GPG (GNU Privacy Guard) to narzędzie do szyfrowania i podpisywania danych, które jest oparte na standardzie **OpenPGP**. Działa na zasadzie kryptografii asymetrycznej, co oznacza, że używa pary kluczy: **publicznego**, który może być udostępniany innym, i **prywatnego**, który jest znany tylko właścicielowi. Klucz publiczny służy do szyfrowania wiadomości, które może odszyfrować jedynie właściciel odpowiadającego mu klucza prywatnego. GPG jest powszechnie stosowane do **bezpiecznej komunikacji** i zapewnienia autentyczności dokumentów poprzez **cyfrowe podpisy**.

Referencje

- Szyfry klasyczne: http://ekryptografia.pl/kryptografia/szyfry-klasyczne/
- Algorytm md5 zapisany w języku Python: https://www.cs.bu.edu/~goldbe/teaching/HW55814/static/pymd5.py
- Informacje o funkcjach skrótu: https://cyberwiedza.pl/funkcja-skrotu-kryptograficznego/
- różnice pomiędzy AES a DES: https://www.geeksforgeeks.org/difference-between-aes-and-des-ciphers/